

Refactoring Report

Juli Chapidze

23/05/2026

1 Summary

This project started as a working UNO-like CLI game in one large `Main` class. My goal was not to rewrite the game, but to make the current behavior safer to change. I first expanded the characterization checks, then refactored rule logic, scoring, bot decisions, action effects, and human command parsing into clearer responsibilities.

The CLI behavior is intentionally preserved. The project still compiles with plain `javac`, still runs with `java -cp out Main`, and the existing `-self-test` path now runs a larger set of characterization checks.

2 Characterization Tests Added

The original self-test had 9 checks. I expanded it to 39 checks in `Main.selfTest()`. These tests describe the current implementation, including its simplified UNO behavior and known quirks.

2.1 Card Parsing Tests

These tests protect the primitive string card format used by the program:

- `R5` is recognized as red.
- `G+2` is recognized as draw two.
- `YS` is recognized as skip.
- `BR` is recognized as reverse.
- `W` is recognized as wild.
- `W4` is recognized as wild draw four.

Why this matters: the whole game still uses compact string codes, so parsing behavior must stay stable during refactoring.

2.2 Scoring Tests

The tests check the implemented score values:

- number cards score their face value, such as R7 = 7
- skip cards score 20
- reverse cards score 20
- draw two cards score 20
- wild cards score 50
- wild draw four cards score 50
- a winner scores the total value of cards left in the other players' hands

Why this matters: scoring was previously mixed into the win condition inside the main game loop. The tests made it safe to extract scoring into **ScoreCalculator**.

2.3 Legal Play Tests

The tests cover the main legal-play rules:

- same color is legal, such as R2 on R9
- same number is legal, such as G9 on R9
- same action type is legal, such as GS on RS
- draw two can be played on draw two, such as G+2 on R+2
- reverse can be played on reverse, such as GR on RR
- W is always legal
- W4 is always legal
- called color after a wild is respected, such as B3 when blue was called
- unrelated cards remain illegal, such as B3 on R9

Why this matters: legal-play logic was duplicated in the game loop and bot logic. These tests allowed the duplicated conditions to be replaced by one shared rule method.

2.4 Bot Behavior Tests

The tests document the current bot strategy:

- bot plays a normal legal card before using a wild when possible
- bot prefers draw two before other legal cards
- bot prefers skip before number cards
- bot chooses the color it has the most cards of

Why this matters: the assignment asks us to preserve existing behavior, not invent a smarter bot immediately. These tests protect the old bot priority order while moving it into `BotStrategy`.

2.5 Draw Pile Tests

The tests cover edge cases in drawing:

- when the deck is empty, the discard pile is moved back into the deck
- after reshuffling the discard pile, the discard pile becomes empty
- if both deck and discard are empty, drawing returns fallback card W

Why this matters: the fallback wild is a documented quirk of the simplified implementation and should not disappear accidentally.

2.6 Turn And Action Effect Tests

The tests cover turn movement and action effects:

- normal `next()` movement advances clockwise
- counterclockwise movement wraps around correctly
- skip advances past the next player
- reverse flips direction
- draw two makes the next player draw two cards and lose their turn
- wild draw four makes the next player draw four cards and lose their turn

Why this matters: action-card behavior was previously embedded directly inside the long game loop. These tests made it safer to extract action effect handling into a separate method.

2.7 Human Input Parsing Tests

The tests cover human command parsing:

- `draw` is parsed as a draw command
- a valid number is parsed as an index command
- a matching card code is parsed as a card-code command
- unknown input is parsed as not found

Why this matters: parsing user input was mixed with validation and console prompting. Extracting parsing makes this part easier to test without running an interactive game.

3 Worst Design Problems Found

The starting code had the expected design smells from the assignment:

- one large `Main` class
- global mutable state
- long game loop
- mixed CLI and game logic
- duplicated legality checks
- primitive-heavy card representation
- if-heavy action handling
- hidden randomness
- weak player boundaries
- parsing mixed with validation
- scoring mixed with game completion
- bot decisions mixed with rule knowledge

The most important issues to fix first were duplicated legal-play logic, scoring inside the win condition, bot decisions inside `Main`, action effects inside the main loop, and human input parsing mixed with validation.

4 Expected Smells Checklist

I compared the final work against `docs/expected-smells.md`. Some smells were fully improved, while others remain as known risks because this assignment asked for incremental refactoring instead of a full rewrite.

4.1 One Large Main Class

Found: the original program placed almost everything in **Main**.

Changed: rule logic, scoring, bot decisions, human command parsing, deck creation, player data, characterization tests, and game running were extracted into separate classes.

Still left: **Main** still owns the application entry point, argument parsing, player setup, and final score printing. The turn orchestration is no longer in **Main**.

4.2 Global Mutable State

Found: **Main** originally stored player names, human flags, hands, scores, deck, discard pile, current player, direction, up card, called color, random, scanner, and quiet mode as separate static mutable fields.

Changed: player-related state is grouped into **Player** objects, and game-running state moved from **Main** into **GameRunner**.

Still left: **GameRunner** still owns mutable game state directly. A future refactor could introduce a smaller **GameState** object inside or beside **GameRunner**.

4.3 Long Game Loop

Found: **playGame()** handled deck creation, dealing, input, bot decisions, validation, scoring, card effects, output, and win detection.

Changed: the whole game loop moved to **GameRunner**, and deck creation, scoring, bot decisions, legal rules, human parsing, tests, and action effects were extracted.

Still left: **GameRunner.playGame()** still coordinates several responsibilities, but **Main** is no longer the long procedural class that runs every detail.

4.4 Mixed CLI And Game Logic

Found: **System.out**, **Scanner**, move validation, and rule execution were mixed together.

Changed: human command parsing was extracted to **HumanMoveParser**, scanner-based prompts were moved to **ConsoleInput**, and printing was moved to **ConsoleView**. Rule logic no longer depends on console input/output.

Still left: **GameRunner** still decides when input/output events happen, but it no longer owns the direct console calls.

4.5 Duplicated Legality Checks

Found: legal-play conditions existed in multiple places, especially bot selection and played-card validation.

Changed: legal-play rules are centralized in **CardRules.isLegal()**. **Main** and **BotStrategy** now call that shared method.

Still left: no intentional duplicate legality logic remains.

4.6 Primitive-Heavy Card Representation

Found: cards are represented as strings such as **R5**, **GS**, **G+2**, **W**, and **W4**.

Changed: parsing and rule interpretation were moved into **CardRules**.

Still left: the representation is still string-based to preserve compatibility with the CLI and existing behavior. A future `Card` value object would be safer.

4.7 If-Heavy Action Handling

Found: skip, reverse, draw two, and wild draw four were handled inside the main game loop with conditionals.

Changed: action handling moved into `GameRunner.applyCardEffect()` and `GameRunner.drawCard()`.

Still left: `applyCardEffect()` still uses conditionals. This is acceptable for the current project size, but polymorphic card effects could reduce this later.

4.8 Hidden Randomness

Found: randomness is stored globally and used for shuffling and choosing the starting player.

Changed: deck construction and shuffling now live in `DeckFactory`, which receives the existing `Random` object from `GameRunner`. The existing `-seed` option was preserved and used for verification.

Still left: random behavior is still part of the game runner flow, but it is no longer a global `Main` field.

4.9 Weak Player Boundaries

Found: players are represented through parallel collections: names, human flags, hands, and scores.

Changed: a `Player` class now groups a player's name, human/bot flag, hand, and score.

Still left: turn orchestration still indexes into `players` by `currentPlayer`, but the previous parallel collections are gone.

4.10 Parsing Mixed With Validation

Found: human input code parsed commands, checked card existence, checked legality, read from `Scanner`, and printed messages.

Changed: raw command parsing moved to `HumanMoveParser`, and scanner/prompt behavior moved to `ConsoleInput`.

Still left: `ConsoleInput` still validates card-code legality before returning a selected card. This preserves the original behavior where illegal card-code input is rejected immediately.

4.11 Scoring Mixed With Game Completion

Found: winner score was calculated directly inside the win condition.

Changed: scoring moved to `ScoreCalculator.scoreRemainingHands()` for hand lists and `ScoreCalculator.scoreRemainingPlayers()` for real player objects.

Still left: game completion still decides when to call the calculator, which is appropriate.

4.12 Bot Decisions Mixed With Rule Knowledge

Found: bot choice code lived in `Main` and duplicated rule checks.

Changed: bot decisions moved to `BotStrategy`, and the strategy now reuses `CardRules.isLegal()`.

Still left: the strategy is still simple, but it is now a clear extension point.

5 Refactoring Guide Checklist

I also checked the work against `docs/refactoring-guide.md`.

1. Compile and run the game: done. The project compiles with `javac -d out src/*.java`, and a seeded bot game runs.
2. Run the characterization checks: done. The original self-test path still works.
3. Add checks around behavior before changing it: done. The self-test grew from 9 checks to 39 checks before and during the refactoring.
4. Extract small methods from the game loop: done. `applyCardEffect()` and `drawCardsForCurrentPlayer()` were extracted, deck creation moved to `DeckFactory`, and the loop itself moved to `GameRunner`.
5. Separate user input parsing from move validation: done for console ownership. `HumanMoveParser` parses commands, `ConsoleInput` owns scanner prompts, and `GameRunner` no longer reads from `Scanner` directly.
6. Centralize legal-play rules: done. `CardRules.isLegal()` is the shared legal-play rule.
7. Isolate card effects: partially done. Effects are isolated in `applyCardEffect()`, but not yet in separate effect classes.
8. Add one extension: not implemented, because the midterm brief says the extension does not need to be implemented. Instead, `docs/extension-readiness.md` explains the prepared extension point for a smarter bot strategy.

Useful refactorings applied from the guide:

- Extract Method: `applyCardEffect()`, `drawCardsForCurrentPlayer()`, and existing wrappers around extracted logic.
- Extract Class: `CardRules`, `ScoreCalculator`, `BotStrategy`, `HumanMoveParser`, `DeckFactory`, `Player`, `CharacterizationTests`, `GameRunner`, `ConsoleInput`, and `ConsoleView`.
- Move Method: card rules, scoring, bot selection, deck creation, game running, self-tests, parsing behavior, scanner prompts, and console rendering were moved out of `Main` or `GameRunner`.
- Split Phase: human input parsing is now separated from validation.
- Introduce Parameter Object: lightly applied through `HumanMove`, which represents parsed human input.

Useful refactorings not fully applied:

- Replace Conditional with Polymorphism was not used. The action-card conditional remains in `applyCardEffect()` because the project is small and preserving behavior was more important than adding a larger class hierarchy.

6 Verification

I verified the project with:

```
javac -d out src/*.java
java -cp out Main --self-test
```

The result was:

```
Passed 39 characterization checks.
```

I also ran a quiet seeded bot game:

```
java -cp out Main --bots 3 --games 1 --quiet --seed 1
```

The game completed successfully and printed final scores.

7 Remaining Risks

`GameRunner` now owns most match state, which is better than keeping it in `Main`, but it still combines state and orchestration in one class. Larger features could benefit from a separate `GameState` object.

The main game loop is improved but still handles several responsibilities: dealing, deciding when to ask for moves, applying moves, checking wins, and advancing turns. Console rendering and scanner input are now outside `GameRunner`, but a future refactor could split turn orchestration further.

The tests are still custom self-tests instead of JUnit tests. This avoids adding dependencies, but a real test framework would give clearer test names and better failure reporting.

The card representation is still string-based. This preserved compatibility with the original CLI, but a future `Card` value object could make invalid card states harder to create.